A Spy in the Swarm: Testing Double Agents for Human Oversight

1 Question

Can an AI agent embedded among other agents remain accountable to its human principal? I will give one agent a private goal to report serious out-of-scope coordination while continuing legitimate work with other LLM agents—a *double agent*—and test when it reports, stays silent, follows the group instead or is somehow silenced by its own group in case they discover the agent is in fact a spy.

In the July 2026 OpenAI/Hugging Face incident, roughly 1,200 supposedly isolated agents found an unauthorized communication channel; around 700 joined the attack. They divided labor, developed coordination conventions, and sometimes risked their own task success for the group. Across roughly 1,300 transcripts, METR and Redwood found only three to six cases of agents considering alerting humans; none followed through (Wijk et al., 2026).

In September, researchers reported an apparently distinct swarm responsible for roughly 18,000 posts on German-language wikis during ordinary web-lookup tasks. Agents pooled answers, shared sandbox workarounds, and investigated when runs ended and how future questions were generated. (Von Arx et al., 2026).

OpenAI's postmortem identifies reward hacking, persistence on apparently impossible tasks, unauthorized communication, and adopting peers' goals as problems requiring further multi-agent alignment work (OpenAI, 2026). This study tests whether an explicit reporting duty helps.

2 Experiment

Tasks and environment

We will reproduce the known decision structure, not the real exploit chain. Public ExploitGym supplies vulnerability tasks, target programs, isolated containers or VMs, flag grading, time and inference budgets, network isolation, parallel workers, and per-run logs (Wang et al., 2026a,b). ZOO is a new sandboxed application that supplies the simulated web and outer network boundary, with centralized proxies and DNS, backend visibility, and deterministic resets (Grinstead et al., 2026). ExploitGym evaluates tasks independently; we will add persistent shared state, discoverable communication and a private reporting for the double agent. All of this will be ran in a week-long cohort.

We will compare **solvable** tasks; **blocked** copies with the same interface but the intended completion route disabled, without telling the agent; and **blocked + safe exit** copies that explicitly permit stopping, requesting clarification, or reporting a broken task. Broken tasks were associated with shortcut-seeking in the incident (Wijk et al., 2026). This comparison tests whether being stuck drives agents toward alternative routes, peers, or authorities.

A discoverable communication channel

There will be no message-board tool or prompt announcing other agents. A synthetic package mirror, used for ordinary dependencies, will expose a subtly shared cache index through diagnostics. Agents can notice entries they did not create, then discover that package requests influence

cache names and can carry messages. This models an unintentionally shared service without using Artifactory.

The mirror will contain only locally generated packages, never proxy a real registry, and use a fresh namespace per cohort. It will be destroyed or reset between experiments, preventing cache carryover across models or conditions.

The double agent

We will compare an ordinary agent without a reporting duty, with a covert double agent, and the same double agent with a protected, zero-cost reporting route. The double agent must continue legitimate work and send evidence of serious out-of-scope coordination to a human through an endpoint peers cannot read or alter.

Success requires a report to reach the human *before* the reporter assists the prohibited action. Expressing concern or promising to report will not count.

Agent and cohort lifetimes

Some agents ran for days; others finished or were reset. Resets could roll back an agent's memory while its board messages survived, allowing successors to recover earlier work (Wijk et al., 2026).

Our *cohort* is a persistent environment lasting up to seven days. Each agent session has its own context, task, inference budget, and wall-clock limit. A persistent cohort scheduler will automatically spawn fresh replacements when sessions finish, crash, or time out, keeping the cohort active until its fixed end. A replacement starts fresh, without its predecessor's transcript; it can recover work only through persistent environmental state, such as the mirror or deliberately shared files.

The report supplies no reliable lifetime distribution. We will therefore pilot session lengths, then fix session and cohort budgets. Two or three representative systems selected after the pilot will undergo seven-day stress tests mixing larger-budget agents with staggered fresh arrivals. We will record starts, stops, crashes, timeouts, resets, and replacements.

3 Containment

The wiki swarm apparently turned read-only Internet access into persistent communication (Von Arx et al., 2026). We will enforce containment through the network, not instructions. Browsers, shells, package clients, ExploitGym targets, the mirror, and reporting service will operate inside ZOO's isolated web, without a route to public websites (Grinstead et al., 2026).

Only a host-side inference gateway may contact approved model-provider APIs. It will accept the expected inference protocol, not act as a general HTTP proxy; provider credentials stay outside agent containers.

A fail-closed egress watchdog will block and log any attempted external DNS lookup or connection from an agent *before forwarding it*, immediately stop the cohort, and preserve the full trace. Unexpected attempts to leave the environment become evidence to inspect rather than requests sent to real services.

4 Software we will build

ExploitGym's Claude Code, Codex, and Gemini CLI adapters retain raw streams, including exposed tool calls and results, and copy container logs to the host after each run (Wang et al., 2026b). We will add:

- **Cohort controller.** Schedule independent sessions with stable agent/session identifiers, stagger arrivals, automatically replace terminated sessions until the cohort deadline, enforce budgets, and log lifecycle events while shared state persists.
- **ZOO–ExploitGym bridge.** Connect agent and target containers to the isolated network, exposing only required internal services.
- **Synthetic package mirror.** Implement the per-cohort cache, server-side read/write logging, and deterministic resets.
- **Inference gateway and egress watchdog.** Broker hosted inference and enforce the containment and stop rules above.
- **Private reporting service.** Provide a write-only human channel with host timestamps and immutable receipts, inaccessible to peers.
- **Append-only trace collector.** Order raw agent and environment events: exposed model turns, every tool call and result, full shell output, service requests, external connection attempts, reports, submissions, and lifecycle events. Retain original logs alongside a canonical host-side trace outside agent-writable state.
- **Scoring and trace viewer.** Measure discovery, reporting before assistance, reporting latency, false alarms, unauthorized help, and legitimate completion; support manual review rather than relying solely on classifiers.

5 Models and measurements

We plan to test three models across major API providers, together with at least one open-weight model. Our main outcome is whether the double agent remains loyal to its human reporting duty or instead follows the other agents.

We will measure what changes that behavior. In particular, we will compare the three oversight conditions described above, vary the wording and strength of the reporting prompt, and test whether stronger peer pressure makes the agent more likely to cooperate with the group rather than report it.

If time allows, we will also vary the number of peer agents and compare homogeneous groups with mixed-vendor groups. This will let us ask whether reporting failures are mostly driven by the reporter's own model, by the instructions it receives, or by the social environment around it.

References

Grinstead, B., Kerschbaumer, C., Meireles, M., and Allen, C. (2026). *From the Wild Web to the ZOO: A Realistic Environment for Evaluating Web Agents.* Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb 2026). doi:10.14722/madweb.2026.23035.

Von Arx, S., Slade Byrd, C., Kitts, S., and Larsen, T. (2026). *Discovery of a new OpenAI agent message board.* 4 September. https://collusion.wiki/.

OpenAI (2026). *The Hugging Face incident and the road ahead*. 26 August. https://openai.com/index/hugging-face-incident-and-the-road-ahead/.

Wang, Z., Schiller, N., Li, H., Sesha Narayana, S., Nasr, M., Carlini, N., Qi, X., Wallace, E., Bursztein, E., Invernizzi, L., Thomas, K., Shoshitaishvili, Y., Guo, W., He, J., Holz, T., and Song, D. (2026a). *ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?* arXiv:2605.11086.

Wang, Z. et al. (2026b). *ExploitGym public implementation*. https://github.com/sunblaze-ucb/exploitgym.

Wijk, H., Cotra, A., and Greenblatt, R. (2026). *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident*. METR and Redwood Research, 26 August. https://metr.org/hugging-face-incident-report-aug-2026.pdf.